Nano Express

INCLUSIVE PUBLISHING TRUSTED SCIENCE

PURPOSE-LED PUBLISHING™

PAPER • OPEN ACCESS

Machine learning-based model for the intelligent estimation of critical heat flux in nanofluids

To cite this article: Shahin Alipour Bonab and Mohammad Yazdani-Asrami 2024 *Nano Ex.* **5** 025012

View the article online for updates and enhancements.

You may also like

- Electronic properties and primarily dissociation channels of fluoroethane compounds
Toshio Hayashi, Kenji Ishikawa, Makoto Sekine et al.

- Characterization of the density and mechanical properties of corn husk fiber reinforced polyester composites after exposure to ultraviolet light
Nasmi Herlina Sari, Suteja, R A Ilyas et al.

- Experimental study of thermal limits with one-side heated smooth channel for plasma facing component safety
Ji Hwan Lim and Minkyu Park

This content was downloaded from IP address 159.226.142.5 on 03/03/2026 at 07:02

NANO EXPRESS

OPEN ACCESS

RECEIVED
4 February 2024

REVISED
17 April 2024

ACCEPTED FOR PUBLICATION
1 May 2024

PUBLISHED
15 May 2024

Original content from this work may be used under the terms of the Creative Commons Attribution 4.0 licence.

Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.

PAPER

Machine learning-based model for the intelligent estimation of critical heat flux in nanofluids

Shahin Alipour Bonab and Mohammad Yazdani-Asrami

Propulsion, Electrification & Superconductivity Group, Autonomous Systems and Connectivity division, James Watt School of Engineering, University of Glasgow, Glasgow, G12 8QQ, United Kingdom

E-mail: mohammad.yazdani-asrami@glasgow.ac.uk

Keywords: artificial intelligence, heat transfer, nanoparticles , neural networks , pool boiling

Abstract

The rising demand for advanced energy systems requires enhanced thermal management strategies to maximize resource utilization and productivity. This is quite an important industrial and academic trend as the efficiency of energy systems depends on the cooling systems. This study intends to address the critical need for efficient heat transfer mechanisms in industrial energy systems, particularly those relying on pool boiling conditions, by mainly focusing on Critical Heat Flux (CHF). In fact, CHF keeps a limit in thermal system design, beyond which the efficiency of the system drops. Recent research materials have highlighted nanofluids' superior heat transfer properties over conventional pure fluids, like water, which makes them a considerable substitution for improving CHF in cooling systems. However, the broad variability in experimental outcomes challenges the development of a unified predictive model. Besides, Machine Learning (ML) based prediction has shown great accuracy for modeling of the designing parameters, including CHF. Utilizing ML algorithms—Cascade Forward Neural Network (CFNN), Extreme Gradient Boosting (XGBoost), Extra Tree, and Light Gradient Boosting Method (LightGBM)— four predictive models have been developed and the benchmark shows CFNN's superior accuracy with an average goodness of fit of 89.32%, significantly higher than any available model in the literature. Also, the iterative stability analysis demonstrated that this model with a 0.0348 standard deviation and 0.0268 mean absolute deviation is the most stable and robust method that its performance minorly changes with input data. The novelty of the work mainly lies in the prediction of CHF with these advanced algorithm models to enhance the reliability and accuracy of CHF prediction for designing purposes, which are capable of considering many effective parameters into account with much higher accuracy than mathematical fittings. This study not only explains the complex interplay of nanofluid parameters affecting CHF but also offers practical implications for the design of more efficient thermal management systems, thereby contributing to the broader field of energy system enhancement through innovative cooling solutions.

1. Introduction

Energy systems are of paramount importance in modern societies as they manage the production, transfer, distribution, and conversion of energy in a way that is beneficial for human life. The enhancement of these systems is crucial to reducing energy waste, making the energy conversion process more efficient, and better use of available resources. In the field of heat transfer study, convection, which is a means of heat transfer between fluids and solid surfaces, has various applications among these systems. Therefore, to optimize the efficiency of energy systems, it is essential to focus on enhancing the heat transfer capabilities of the working fluids in these systems. Understanding the dynamics of heat transfer, particularly in thermal systems such as heat exchangers, plays a key role in managing both temperature and energy transfer [1–5]. In such applications, where the coolant fluid is used, it is preferred the system is designed to work in pool boiling conditions due to its lower temperature change and high heat absorption [6]. The reason behind the higher heat absorption is that in most cases, the

© 2024 The Author(s). Published by IOP Publishing Ltd

latent heat of the fluids is much higher than their heat capacity during a constant volume or constant pressure conditions.

In the study of heat transfer, pool boiling is a heat transfer process where a pool of bubbles is created in the contact surface with a heat source while the fluid is relatively stationary [7]. When the boiling process happens, at first, the nucleate boiling happens, where vapor bubbles form at nucleation sites on the heated surface. At this condition, the bubbles detach from the hot surface and move within the fluid, which results in an effective heat transfer [8]. By raising the input heat flux to the fluid, the creation of the bubble increases till a situation where a continuous creation of bubbles makes a layer of vapor that covers the heating surface [9]. This condition is known as film boiling and the amount of heat flux that causes this transition between the boiling conditions is known as Critical Heat Flux (CHF) [10]. In other words, CHF is the maximum heat flux that one surface can handle before experiencing a transition from nucleate boiling to film boiling [11]. When the heat transfer system, especially cooling systems, cannot dissipate the produced heat load effectively, a temperature increase happens, leading to a thermal runaway, failure, or low-efficiency condition [12, 13]. By accurately predicting CHF under various conditions, our model aids in the design and optimization of cooling systems, ensuring they operate within safe thermal margins while maximizing heat dissipation. Also, the developed model's capability to predict CHF with high accuracy for nanofluids of varying nanoparticle sizes and types allows for the tailored design of nanofluids. This design flexibility ensures that the selected nanofluid composition is ideally suited to the specific requirements of the application, thereby leveraging the full thermal potential of nanofluids.

As a solution, nanofluids, which generally have higher heat conductivity than pure fluids, can be used instead of the common commercial coolants to enhance the overall heat transfer of the system through CHF increase [14]. Therefore, the advancement in heat transfer systems through using innovative technologies like nanofluids has been promoted in recent decades. Drawing inspiration from the works of Datta and Halder [15–19], which explored nanoparticle size effects in jet impingement cooling using oxide–water-based nanofluids, our research contributes to this burgeoning field by offering a robust, predictive model that captures the complex interactions of different parameters influencing CHF in nanofluids. By conducting experiments, the properties of different nanofluids and their effects on the CHF have been investigated in the literature. However, as there are a range of factors that can impact the CHF of nanofluids, simple theoretical equations cannot evaluate the actual CHF and heat transfer of these fluids for real applications. Because of this, a novel approach rather than these ways seems to be necessary to reach an acceptable prediction accuracy.

For the prediction of CHF, there are several numerical approaches. One of these methods is Calculative Fluid Dynamics (CFDs) in which the fundamental equations will iteratively calculated for a meshed domain. Although many advancements in these methods have been made and their accuracy has been improved over the two decades, they have some fundamental drawbacks, like high computational costs. As an alternative, using intelligent techniques, like Artificial intelligence (AI) and machine learning (ML) methods, for the estimation of parameters that are useful in engineering designs, has gained popularity in recent years [20–31]. Naturally, ML-based models offer several pros over CFD simulations and traditional mathematical fitting methods, particularly in terms of accuracy, generalizability, rapid response, and retraining ability. These advantages make ML models highly suitable for complex systems like critical heat flux (CHF) estimation in nanofluids, where the interactions between the effective input parameters (e.g. temperature, nano-particle size, etc.) and the value of CHF are intricate, and the data is vast.

ML models can achieve high levels of accuracy in predicting outcomes because they can learn from large datasets and identify complex nonlinear relationships that are difficult to model explicitly through CFD or mathematical fittings. Unlike traditional methods that may rely on simplifications or assumptions that limit accuracy, ML models can directly learn from experimental or simulated data, capturing the underlying physics more faithfully [32, 33]. ML models, once trained, can apply learned patterns to predict outcomes under conditions not explicitly covered in the training data. This generalizability is particularly valuable in engineering applications where conditions may vary widely. In contrast, CFD simulations are specific to the boundary conditions and parameters defined at the onset; any change requires a new simulation. Mathematical fittings, while useful, often struggle to adapt to data outside the range for which they were developed. Once trained, ML models can make predictions almost instantaneously, offering fast responses to new and unseen data. This capability is invaluable for designers in various fields of study, especially for real-time applications and for scenarios where quick decisions are necessary [34, 35]. CFD simulations, in comparison, are computationally intensive and time-consuming. The need to run simulations from scratch for new conditions further exacerbates the delay in obtaining results [36]. Moreover, ML models can be updated and retrained with new data, allowing them to improve over time or adapt to changes in the system being modeled. This retraining ability ensures that ML models remain relevant and accurate as more data becomes available or as the system evolves. Conversely, CFD models and mathematical fittings lack this adaptability; any significant change in the system or the discovery of new data necessitates developing a new model or fitting function from the ground up [33]. The only requirement of the ML modelling approach is a reliable set of data, which is usually provided from

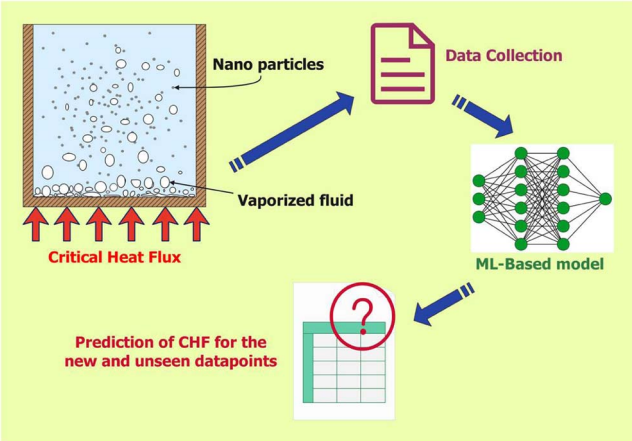

Figure 1. ML modelling steps in estimating critical heat flux of nanofluids in this paper.

experiments. After training the machine with this data and learning the influence of each parameter (see figure 1), the model will be able to predict the target variable for an unseen condition.

There have been some efforts by other researchers to model CHF or other heat transfer properties of the cooling- or nano-fluids. In [21], the Random Forest (RF), Support Vector Regression, and Multilayer Perceptron (MLP) techniques were used to predict the CHF of different nanofluids. The results demonstrated that the MLP model had better accuracy and stability than the other models that were used. In [37], the pool boiling heat transfer coefficient of hybrid nanofluids with TiO_2 and SiO_2 nanoparticles in a water base for different concentrations was experimentally investigated on a copper surface and then a model based on Artificial Neural Networks (ANN) was developed by using the results of the experiment. In [23], a model based on RF was developed to predict CHF of pillar-modified surfaces and assess how it is influenced by different properties of a surface, e.g. roughness and porosity. In an almost similar approach to [21], the input data were collected for their model from published papers in the literature. The results of their study demonstrated that RF was a better technique for their dataset. It was explained that this is due to the sparse number of data points in the experimental dataset and suggested the use of neural networks when more data is collected. In [38], Faridzadeh *et al* focused on enhancing the heat transfer mechanisms in energy systems. Effects of various parameters such as Richardson number, Reynolds number, inclination angle, and solid volume fraction of nanoparticles are examined for their impact on flow and heat transfer. Their study aims to analyze laminar mixed convection in an inclined, square, lid-driven cavity filled with copper–water nanofluid using ANN. In this study, the effects of various parameters such as Richardson number, Reynolds number, inclination angle, and solid volume fraction of nanoparticles are examined for their impact on flow and heat transfer. They used ANNs to derive a relationship involving the independent parameters to calculate the Nusselt number, employing the backpropagation learning algorithm with a tangent sigmoid transfer function. The results of their study highlighted that inclination angle and solid volume fraction significantly affect the fluid flow and heat transfer in the cavity. In terms of the performance of their developed model, a coefficient of determination (R^2) of 0.9999 between the real values and ANN results indicated an excellent predictive performance of the ANN. Also, the maximum error in prediction was less than 0.5829, and the mean square error (MSE) is 5.37×10^{-5}, demonstrating the accuracy of the ANN. In a very useful work regarding the application of AI for nanofluids, the goal of research that was conducted by Rostami S. *et al* [39] was to predict the thermal conductivity of SiO2 in a water–ethylene glycol mixture, using a feed-forward network with the Levenberg–Marquardt learning algorithm based on experimental data for various volume fractions of nanoparticles and different temperatures. The researchers generated experimental data for nanoparticle volume fractions from 0 to 5% and temperatures from 25 to 50 °C. They also proposed an algorithm to identify the optimal number of neurons in the ANN's hidden layer to predict thermal conductivity most accurately. The results of the study showed that the ANN model with 8 neurons in the hidden layer showed the best performance, with a high correlation coefficient of 0.993861 indicating the accuracy of the ANN predictions. Moreover, the surface fitting method had an MSE of 2.7547e-05 and a maximum error of 0.0125, with a correlation coefficient of 0.9919. Relatedly, in [40–44], the authors used AI approaches to predict the CHF of fluids under different conditions.

As part of the application of this study, a thermal solar system incorporating nanofluid has a significant contribution to the advancement of renewable systems. In [45], the authors focused on the experimental study of the viscosity properties of a non-Newtonian nanofluid composed of Multi-Walled Carbon Nanotubes

(MWCNTs), Zinc Oxide (ZnO), water, and ethylene glycol at varying volume fractions and temperatures. The researchers observed that increasing the volume fraction (ϕ) of nanoparticles in the fluid resulted in a more pronounced non-Newtonian behavior. The study measured the viscosity at temperatures ranging from 25 °C to 50 °C and volume fractions from 0.075% to 1.2%. Understanding the rheological properties of nanofluids, including viscosity, is essential because viscosity directly affects the CHF in boiling heat transfer applications. In [46], the authors investigate the heat transfer enhancement in a uniquely designed microchannel heat sink. The heat sink design integrates double-layer channels with sinusoidal cavities and rectangular ribs which promote fluid mixing and enhance heat transfer. The findings of this study demonstrated that the addition of graphene nanoplatelet–sodium dodecylbenzene sulfonate (GNP–SDBS) nanoparticles to the water, at mass fractions ranging from 0 to 0.1%, significantly improved the heat transfer as characterized by the Nusselt number, which increased by approximately 20% across all Reynolds numbers considered (50 to 1000). It was concluded that the novel microchannel design is recommended for applications with Reynolds numbers less than 300. It also was observed that as the Reynolds number and mass fraction of solid nanoparticles increase, the Nusselt number increases, and the heat resistance of the lower wall of the microchannel decreases. Moreover, the innovative design offers substantial heat transfer enhancement while also promoting a more uniform temperature distribution, making it beneficial for micro-scale thermal management applications. In [47], the authors explored the thermal performance of a water-based hybrid nanofluid containing iron oxide (Fe_3O_4) and carbon nanotubes (CNTs) in a wavy-walled trapezoidal enclosure. In this study, various parameters, such as the Darcy number, Hartmann number, volume fraction, undulation number of the wavy wall, and Rayleigh number (thermal buoyancy parameter), are analyzed for their impact on flow and heat transfer characteristics. Their methodology for this study was using of modified Navier–Stokes equations incorporating Lorentz hydromagnetic body force, Darcian, and Forchheimer drag force terms and the Galerkin Finite Element Method (FEM) with COMSOL Multiphysics software is used for the numerical solution. The results of the study highlighted that with increasing Hartmann and Rayleigh numbers, the average Bejan number is significantly reduced, indicating a decrease in entropy generation due to magnetic damping. The average Nusselt number, a measure of heat transfer enhancement, increases notably at high Rayleigh numbers, particularly when the undulation number of the wavy wall is elevated.

Overall, there is a lack of a fast and reliable model for predicting the CHF of different water-based nanofluids under different working conditions. According to the application that this paper is focused on (water-based nanofluids as coolant in heat exchanging systems), we have developed four novel, highly reliable, and extensively stable AI-based models that are able to estimate CHF for water-based nano-fluids under saturation conditions. These models have been developed based on four renowned and robust AI algorithms namely Cascade Forward Neural Network (CFNN), eXtreme Gradient Boosting (XGBoost), Extra Tree, and Light Gradient Boosting (LightGBM). These methods can handle extremely complex, nonlinear relationships and vast datasets with outstanding accuracy. The results of the work demonstrated a significant improvement in the stability of models' prediction and the accuracy of estimation in comparison with experimental data. The developed model with its accurate prediction of CHF can be very useful for the designers of coolant systems to estimate the CHF of their designated system in just several milliseconds of time. Also, it can be used for optimization purposes where the designer can test the behavior of their system under different working conditions and reach an optimal choice. In fact, accurately predicting CHF can lead to the design of more efficient cooling systems, potentially preventing thermal runaway failures in critical applications such as electronic cooling, nuclear reactors, and renewable energy systems. The results of this work could also pave the way for further studies, such as the integration of your predictive models into real-time monitoring systems for thermal management or exploring the effects of other nanoparticle types and concentrations on CHF.

In the following, first, the data collection and processing have comprehensively been provided. Then, the architecture of the ML models and the performance indexes that have been used are explained. Then, in the third section, the results and findings of the paper are discussed. Also, a comprehensive sensitivity analysis of the effective parameters of the CFNN model has been made and the results of the study have been discussed. Finally, a conclusion based on the results is made in the fourth section.

2. Methodology

In this section, a detailed explanation of the data preprocessing approach, the architecture of ML models, and the performance indexes for results comparison have been provided. The rationale behind the selection of these specific ML algorithms can be explained in threefold:

1. The chosen models have a proven track record of handling complex, nonlinear datasets across various domains and have shown excellent accuracy and reliability in previous studies in various engineering fields.

For instance, the CFNN, belonging to the family of artificial neural networks, is renowned for its capability to learn complex relationships between inputs and outputs and indicates a superior performance in various applications ranging from time-series prediction to complex engineering problem-solving. This selection aligns with our objective to build a model that not only fits our experimental data with high accuracy but also possesses the reliability and robustness that have been documented in extensive ML literature.

2. By incorporating ML methods from different families of ML algorithms (neural networks for CFNN and ensemble methods for XGBoost, LightGBM, and Extra tree) this study leverages the distinct strengths of these methodologies for CHF prediction of nanofluids, helping us to reach a more robust model. Neural networks, with their ability to learn complex patterns through layers of interconnected neurons, offer a dynamic modeling capability. On the other hand, ensemble methods like XGBoost and Extra Trees enhance prediction accuracy by combining the outcomes of multiple decision trees, thereby utilizing the collective learning power of several models. This comprehensive approach ensures that our findings are not only based on a singular perspective of machine learning but are validated across a spectrum of advanced ML techniques.

3. Another purpose is to provide a comprehensive comparison between the results of the models of this paper with developed models for CHF prediction in the literature, showing that the selected models have significant improvement in the accuracy Support Vector Regressor, Multilayer Perceptron Neural Network, and Random Forest Regressor for similar dataset [21]. The results of this paper which will be presented in section 3 have demonstrated a superior performance in terms of accuracy and stability (as indicated by higher R-squared values and lower standard deviations compared to the models in [21]), validate these approaches and underscore the effectiveness of these models in advancing the state of the art.

2.1. Data preprocessing

In this paper, a comprehensive experimental dataset for different nanomaterials has been used from the existing literature to be implemented in the ML models. As each of these papers focused on different parameters, which were not necessarily used by others, the data points and effective parameters that were more common among them have been chosen to make a larger, comprehensive, and diverse dataset containing more critically effective parameters. A summary of 168 data points collected during the data collection process has been provided in table 1, which have been used as inputs to the ML models. These nanofluids are water-based and work under saturated conditions. Because of this, only the Pressure can completely represent the thermodynamical condition of the fluid. Therefore, the temperature, which is dependent on pressure in saturation conditions, can be removed from the input dataset. After the dataset collection, which was listed in table 1, it needs to be preprocessed to get ready for being used in the ML models. The initial step is to make a tabular dataset where each row represents an observation while the columns represent the effective parameters. These parameters are namely, pressure, substrate thermal effusively, nanoparticles' thermal effusivity, the volumetric concentration of nanoparticles, and CHF. Then, the dataset needs to be randomly split into training and testing datasets. Using the former dataset, the model will be trained and the performance of the model for accurate prediction will be assessed by using the latter data points. It is common among researchers to consider 70%–90% of all available data for training purposes. In this study, the mean value of this range, which is 80% is considered for training and the rest of the data points for the testing process.

2.2. Structure of the ML models

Every ML technique is developed based on some mathematical equations and is programmed to behave in an optimum way to reach the highest accuracy. Most of these algorithms are inspired by natural samples from the environment. As an example, ANN is a family of AI/ML techniques that try to understand the way that the input data is connected to output data in a manner like a neural system in humans. In this part, a brief introduction of each technique, its important parameters, and their mathematical fundamentals is explained. Moreover, for individual techniques, a figure is provided to help better understand of these complex methods.

2.2.1. CFNN

The CFNN belongs to the family of ANN, a potent method for handling highly non-linear datasets. A CFNN model is made with at least three layers namely input, hidden, and output layer. In each of these layers, there are a number of elements, which are known as neurons [77]. These elements are responsible for performing computations on input data. Each neuron has at least one weight and bias factors that help it to understand the relationship of different parameters with the output data [78, 79]. In the CFNN model, by moving from the input layer towards the output layer, the number of weight factors increases in a cascade manner [80]. In other words, in this approach, the upper hand layers provide feeding lines to the neurons of all the next layers. For

Table 1. Summary of data collection from existing literature.

References	Nanomaterial	Size of nanoparticles [nm]	Volumetric concentration [%]	Pressure [kPa]	Number of data points
[48]	TiO_2	7–250	0.000249–0.0249	98.16	16
	Al_2O_3	15–60			
	ZrO_2	20–45			
	YSZ	25			
[49]	TiO_2	21	0.00094–0.0252	100.4	16
	Al_2O_3	13			
	SiO_2	20			
[50]	Al_2O_3	22.6–46	0.000015–0.0002	99.16	13
[51]	Al_2O_3	60	0.0001–0.01	100.46	11
	Fe_3O_4	25			
	TiO_2	15			
[52]	Al_2O_3	160	0.001–0.1	101.33	9
	ZrO_2	180			
	SiO_2	30			
[53]	Al_2O_3	50	0.001–0.1	3	8
[54]	SiO_2	10	0.1–2	100.92	8
[55]	Al_2O_3				
	Fe_3O_4	25	0.0001	100.46–1100	6
[56]	SiC	100	0.0001–0.01	100.46	6
[57]	TiO_2	85.3	0.000001–0.1	96.3	6
[58]	TiO_2	85	0.0001–0.1	101.33	5
[59]	Al_2O_3	30	0.076–0.382	94.61	5
[60]	Al_2O_3	45	0.00001–0.1	4	5
[61]	Al_2O_3	45	0.01–0.09	101.33	5
[62]	Al_2O_3	45	0.00001–0.1	4	5
[63]	Al_2O_3	40	0.001–0.1	84.6	5
[64]	Al_2O_3	10	0.01–0.05	97.66	4
[65]	Al_2O_3	30–50	0.0025	70–130	4
[66]	Al_2O_3	47	0.5–4	100.46	4
[67]	SiO_2	20	0.01–0.1	83	3
[68]	SiO_2	10	0.005–0.01	83.8	3
[69]	SiO_2	10.5	0.001–0.005	84.6	3
[70]	Cu	10	0.028–0.113	98.26	3
[71]	Cu	15	0.028–0.113	98.26	3
[72]	SiO_2	100	0.00034–0.0013	99.61	3
[73]	TiO_2	45	0.00025–0.00075	101.33	3
[74]	SiO_2	30	0.00627	7.4–103	3
[75]	Al_2O_3	50	0.01–0.03	100.27	2
[76]	Al_2O_3	47	0.001	97.9	1

instance, as can be seen in figure 2, the input layer provides additional feeding lines (shown by red arrows) to all the subsequent layers. Therefore, the total number of computational elements has risen in comparison with the traditional ANNs, where each neuron has only one weight factor. As a result, this technique requires more time to get trained. Although this approach makes the model extremely complex, it indeed gives the ability for the model to better understand the way the parameters are related to each other [81]. So, it can handle even more complex datasets where there are a lot of input parameters and data points or where the data is very non-linear.

The way that the model updates the weight and bias factors is by using optimization/training functions. There are different algorithms for these functions [82]. Once the updating of the weight factors of a neuron is done, another type of usually non-linear function known as the activation function will decide whether the results of this neuron are being fed to the next layer [83]. Further information on these functions is provided in section 3.1.

The mathematical representation of CFNN is expressed as equation (1) [84]. In this equation, b represents the forecasted output, x stands for the input feature, and Ψ^p and Ψ_p^H signify the activation functions of the output and hidden layers, respectively.

$$b_p = \Psi^0\left[\sum_{i=1}^{n}\frac{\Psi^i}{\Psi^0}w_i^0x^i + \left(\sum_{j=1}^{n}w_i^0x^i \quad \Psi_j^H\left(\sum_{i=1}^{n}w_{jh}^Hx_i\right)\right)\right] \tag{1}$$

Adding the bias factors to the input and hidden layers changes the formula to equation (2) where w_p^H and w^b represent the weights from bias to the hidden layer and output layer, respectively. Equation (2) involves weighted

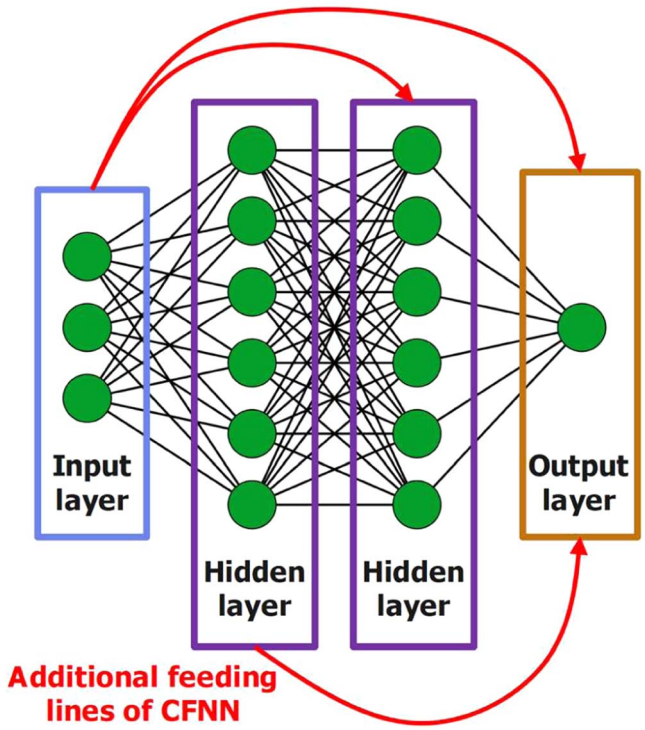

Figure 2. Schematic of a multilayer CFNN model with two hidden layers.

additions and non-linear operations, capturing the intricate computations conducted within the network.

$$b_p = \Psi^0\left[\sum_{p=1}^{P}\frac{\Psi^p}{\Psi^0}w_p^0x^p + \left(w^b + \sum_{j=1}^{P}w_p^0x^p\Psi_q^H \quad \left(w_p^0 + \sum_{p=1}^{P}w_{qh}^Hx_p\right)\right)\right] \tag{2}$$

2.2.2. XGBoost

XGBoost is a member of the family of gradient-boosting methods. It can handle both classification and regression problems. It is very popular among researchers since it can handle very non-linear datasets and demonstrate good performance in most of cases [85, 86]. In fact, it is an ensemble ML algorithm that makes an estimation by the combination of the results of multiple weak learners like the decision tree method [87]. Figure 3 demonstrates how the XGBoost combines the predicted results of subtrees using a weighted average. The simple mathematical expression of this ensemble model can be written as follows [88]:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i),\ f_k \in R \tag{3}$$

where $\hat{y}_i$ is the estimated value, k is the number of subtrees, f is the correlation that maps the relationship between the structure and weights of a tree in the subtree set, x_i is the input vector, and R represents the subtree set.

By using a sequential approach of learning, each weak model improves the results of the previous one and this process will continue till it reaches the preset amount of error. To reach this target, it uses an objective function and tries to minimize it where this function can be written as follows:

$$Y = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{k=1}^{K}\Omega(f_k) \tag{4}$$

where, Y is the objective function, L is the loss function, and Ω is the regularization term. In equation (4), the first term $\sum_{i=1}^{n} L(y_i,\ \hat{y}_i)$ evaluates the error between the estimated and actual values and the second term $\sum_{k=1}^{K}\Omega(f_k)$ quantifies the complexity of the tree model. The complexity terms are of paramount importance as not only control the complexity of the model but also prevent overfitting conditions by smoothing the weights among the leaf nodes. The term leaf refers to the predicted value of a single decision tree. The mathematical equation can be written as follows:

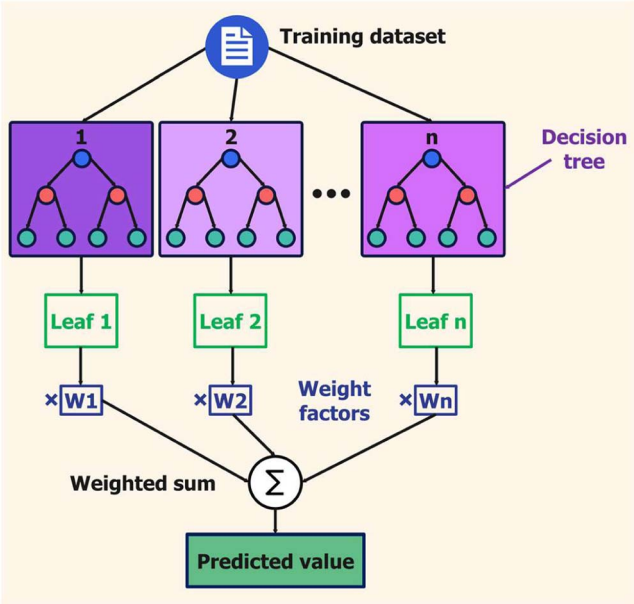

Figure 3. Schematic of XGBoost technique.

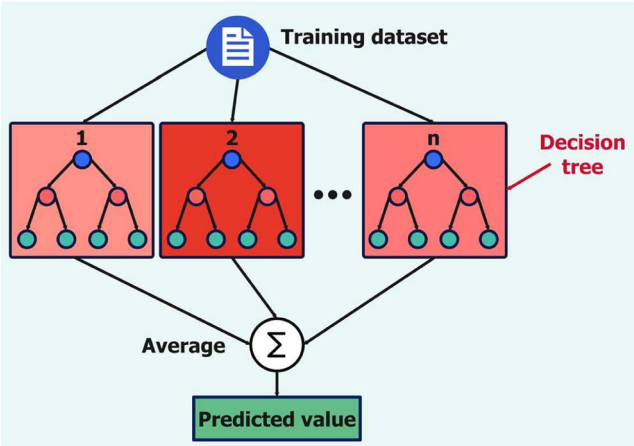

Figure 4. A schematic of the Extra Tree regressor method.

$$\Omega(f_k) = \gamma P + \frac{1}{2}\lambda \sum_{j=1}^{P} \omega_j^2 \tag{5}$$

where, P is the number of leaf nodes, ω is the weight of each leaf, and γ and λ are hyperparameters, helping to avoid overfitting.

2.2.3. Extra tree

Extra trees or Extremely Randomized Trees is a method that uses a vast number of decision trees to reach a more accurate prediction. Unlike XGBoost, it does not consider different weights for the results of each tree and uses simple mean values as shown in figure 4. The way it works is remarkably similar to the Random Forest (RF) method. Like RF, Extra Trees builds multiple decision trees, but it differs in the way that it constructs each tree. Actually, the Extra tree algorithm differs from the RF algorithm in two main ways [89], (i) the Extra tree randomly selects a binary splitting value to divide decision tree nodes, whereas the RF employs an optimal splitting criteria value. (ii) The extra tree minimizes deviation using the full set of data points, while the RF uses bootstrap, a sampling method with replacement. Additionally, the Extra tree offers two advantages in comparison with RF and decision tree-based algorithms [90], (i) its node splitting process is straightforward and

more effective than other ensemble learning methods. (ii) Extra tree addresses overfitting issues through its high randomness and utilization of the entire dataset sample to minimize deviation.

Assuming that there is a D-dimensional input space and real-valued outputs, we can write [91]:

$$x \in R^D, \quad y \in R \tag{6}$$

Consider a set of training data $\{(x_n, y_n)\}_{n \in S}$ for a specific node in a tree, with S representing the set of sample indexes for that node.

The tree-building process follows these steps: Initially, a random selection is made of K input dimensions. For each chosen dimension d, a random binary splitting value c is selected. This value classifies data points 'n' in 'S' based on whether $x_{n,d} < c$ (assigned to the left branch, denoted as L) or $x_{n,d} \geqslant c$ (assigned to the right branch, denoted as R). This step differs from RF which employs optimal splitting criteria for c.

Subsequently, the algorithm calculates the score for each dimension 'd' and its splitting value using the equation (7):

$$s_d = |L|\, f_{score}(y_L) + |LR|\, f_{score}(y_R) \tag{7}$$

where $|L|$ and $|R|$ represent the number of data points in the left and right branches, y_L and y_R denote the y values in the left and right branches, respectively.

Next, for the binary classification function f_{score} is the Gini index given by:

$$f_{score}(y) = 1 - (p_{-1}^2 + p_1^2) \tag{8}$$

where p_{-1} is the proportion of samples with $y = -1$ and respectively, p_1, is the proportion with y = 1. For regression problems, the score function is the negative value of the variance of data points. Therefore, it can be defined as equation (9):

$$f_{score}(y) = -\frac{1}{n} \sum_{i=1}^{n} (y_i - mean(y))^2 \tag{9}$$

Finally, it selects the dimension with the highest score s_d, keeps its split in the node, and recursively calls these steps on the two subtrees made out of L and R, respectively. This recursive process continues until a minimum node size (N_{leaf}) is attained. At this point, a leaf node is created, storing the most frequent output value in the case of classification and the average value in the case of regression. This entire process is repeated for each tree until completion.

2.2.4. LightGBM

LightGBM is a popular boosting method that overcomes the limitations of Gradient Boosting Regression (GBR) when dealing with large datasets. It employs two key techniques: gradient-based one-side sampling (GOSS) and exclusive feature bundling (EFB), to enhance training efficiency without sacrificing accuracy or performance. GOSS focuses on data instances with larger gradients, considering them more informative. It selects a subset A containing the top $a \times 100\%$ of instances with the highest gradients. For the remaining instances, subset B is created through random sampling from the lesser gradients group, making the process efficient and focused on significant data. The variance gain calculation, used for data splitting, prioritizes these subsets to maintain information integrity while reducing computation. Equation (9) explains how the variance gain $\tilde{V}_j(d)$ is calculated, factoring in the gradients from subsets A and B, and adjusting for the sampling rate. This approach enables precise and effective data splitting by emphasizing instances that contribute most to learning.

$$\tilde{V}_j(d) = \frac{1}{n} \left(\frac{\left(\sum_{x_i \in A_l} g_i + \frac{1-a}{b} \sum_{x_i \in B_i} g_i\right)^2}{n_l^j(d)} + \frac{\left(\sum_{x_i \in A_r} g_i + \frac{1-a}{b} \sum_{x_i \in B_r} g_i\right)^2}{n_r^j(d)} \right) \tag{10}$$

In this formula, $n_l^j(d)$ *and* $n_r^j(d)$ denote the left and right nodes, respectively, A_l and A_r represent subsets of A, and B_l and B_r are the subsets of B. EFB reduces the number of features by bundling together non-overlapping features in a sparse dataset, boosting computational efficiency and model performance. Furthermore, LightGBM adopts a leafwise tree growth strategy to improve model outcomes and mitigate overfitting, leveraging parameters like minimum child samples and maximum number of leaf nodes. The minimum child samples parameter ensures that leaf nodes have a sufficient number of data points, preventing overly complex models, while the maximum leaf nodes parameter balances the model's depth and complexity, guarding against overfitting but ensuring adequate learning capacity.

2.3. Indicators for performance evaluation

To effectively compare the performance of each method, some common indicators have been chosen due to their popularity among data scientists and ML researchers.

(i) Coefficient of determination (R^2)

Known as R-squared, is an extremely useful tool to assess how well the predicted results fit to the actual values. It has a value between 0 and 1. A value of 1 indicates a perfect fit, while 0 means the model does not explain any variability. It is useful for understanding the goodness of fit and whether the model captures a massive portion of the variability in the data. Its mathematical equation can be written as follows [82]:

$$R^2 = \frac{\sum_{k=1}^{n_s}(a_k - \bar{a})^2 - \sum_{k=1}^{n_s}(a_k - b_k)^2}{\sum_{k=1}^{n_s}(a_k - \bar{a})^2} \tag{11}$$

where $\bar{a_k}$ is the mean value of a_k, b is the predicted value, n_s is the number of samples of the training dataset, and a_k is the actual values.

(ii) Root mean squared error (RMSE)

RMSE is a really useful index as it penalizes larger errors more heavily and is commonly used in regression problems. The square root is applied to make the units of RMSE consistent with the original data. Equation (12) provides its mathematical expression [92]:

$$RMSE = \sqrt{\sum_{k=1}^{n_s} \frac{(a_k - b_k)^2}{n_s}} \tag{12}$$

(iii) Mean absolute relative error (MARE)

MARE is a dimensionless index that measures the average relative difference between predicted and actual values as shown in equation (13). It is especially useful when the absolute scale of the data varies significantly. MARE gives a sense of the average proportional error, helping to understand the model's performance in relation to the magnitude of actual values [82].

$$MARE = \sum_{k=1}^{n_s} \frac{\left|\frac{(a_k - b_k)}{b_k}\right|}{n_s} * 100 \tag{13}$$

(iv) Mean absolute relative deviation (MARD)

MARD is another percentage-based metric that offers insight into the relative variability of the data. It measures the average relative difference between each data point and the mean. MARD can help assess the consistency of the dataset in terms of proportional differences and offers a perspective on the relative dispersion of the values. It is useful to assess how the performance of a model varies in N number of iterations and can be written as follows [93]:

$$MARD = \sum_{j=1}^{N} \frac{\left|\frac{x_j - \bar{x_j}}{\bar{x_j}}\right|}{\mathrm{N}} * 100 \tag{14}$$

In equation (14), x_j is the performance of the model and x_j is the mean value of x_j.

(v) Standard deviation (Std)

The Std measures the amount of variation or dispersion in a set of values. A lower standard deviation indicates that the values are closer to the mean, while a higher standard deviation suggests greater variability. Therefore, similar to the MARD, it is an especially useful index for analyzing the stability of the model [20, 81].

$$Std = \sqrt{\frac{\sum_{j=1}^{N}(x_j - \bar{x_j})^2}{N}} \tag{15}$$

Before starting of modelling section, the process for developing and validating a machine learning-based model for predicting critical heat flux (CHF) using nanofluids needs to be explained. As can be seen in figure 5, the modelling process begins with the collection of available data from the literature on nanofluids and their associated CHF. Then, the gathered data is then split into two sets: training data and testing data. The training data is used to develop the AI model, while the testing data is reserved for evaluating the model's performance.

Next, the training data is fed into the AI algorithms, which in this case include models like CFNN, XGBoost, Extra Tree, and LightGBM, for the model to learn the relationship between input parameters and the CHF. Once

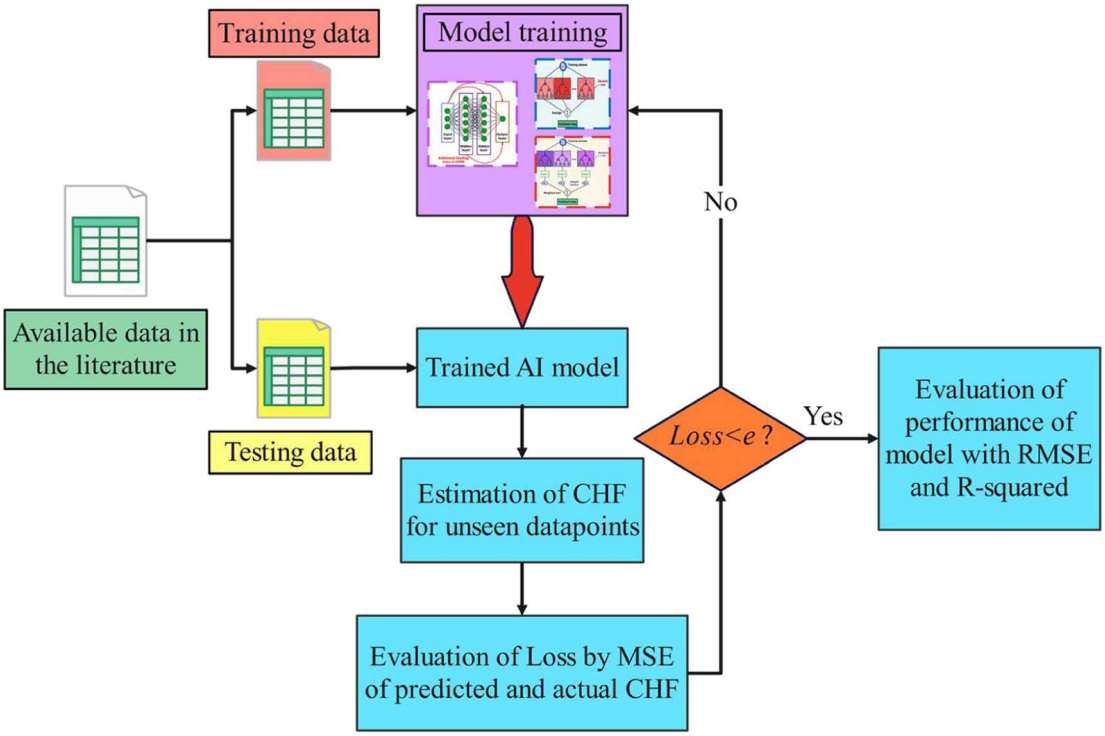

Figure 5. Flowchart of the procedure for model training and evaluation.

the model is trained, it's considered ready for predictions and further evaluations. The trained model is then used to estimate CHF for data points it hasn't seen before, typically the testing data.

Once done, the loss (error) is calculated using a metric like Mean Squared Error (MSE), which measures the average of the squares of the errors between the predicted CHF and actual CHF. If the loss is less than a predefined threshold (e), the model's performance is deemed satisfactory. If the loss is acceptable, the model's performance is further evaluated using metrics such as RMSE (Root Mean Squared Error) and R-squared. These metrics assess the accuracy of the model's predictions. If the loss is not less than the threshold, the model may need further tuning or retraining until the loss becomes acceptable.

3. Results and discussion

In this section, first, the results of each method and their hyperparameter tuning are proposed and explained in detail. Then, a comparison of the performance of the different methods that have been tested in the present work and also the existing model in the literature is made and discussed.

In sub-sections 3.1, 3.2, and 3.3 the controlling parameters of each technique that have a significant effect on the accuracy of the final model are discussed. Then, the way that the models behave with each set of these parameters is tested and their performance is assessed using the indexes that were proposed in the previous section. Finally, the details of the model that performed best are provided.

3.1. CFNN

The number of hidden layers is remarkably effective in the accuracy of the model. If there are insufficient layers, the model may be overly basic and incapable of grasping the essential features in the data, resulting in underfitting. Conversely, too many hidden layers can make the model overly intricate, leading to longer computation times and causing it to pick up on irrelevant details, known as overfitting. Tuning of the hidden layers is useful for understanding how changes in the number of layers affect this generalization capability of the model to predict a target value for an unseen data point. Most of the researchers consider only one hidden layer for their model due to the simplicity of the work. That said, it has been shown in various AI modelling papers in the literature that the models with two or greater number of hidden layers are generally more robust than single hidden layer ones. In this study, the number of hidden layers is considered to vary between 1 to 5.

Another crucial hyperparameter in CFNN is the number of neurons in each layer. In simpler terms, a neuron is a basic processing unit that computes a weighted sum of inputs, adds a bias, and then processes this sum

Table 2. Sensitivity analysis results of the number of hidden layers and neurons for the CFNN model.

Number of Layers	Number of neurons	RMSE	R-squared	MARE [%]	Testing time [ms]
1	30	1204.2764	0.80655	7.688875	7.9218
2	[30 30]	882.4681	0.85758	7.019789	12.2984
3	[20 20 20]	1025.9531	0.84267	8.663572	16.8381
4	[25 25 25 25]	1098.2513	0.83579	14.11152	67.6295
5	**[20 20 20 20 20]**	**672.8318**	**0.86589**	**5.280222**	**18.7263**

Table 3. Performance of the CFNN model using different training functions.

Training function	RMSE	R-squared	Testing time [ms]
LM	**672.8319**	**0.8658**	**18.7263**
SCG	1158.576	0.8005	12.896
FPCG	1260.653	0.8128	23.893
PRCG	2853.848	0.7250	15.933
RB	1056.632	0.8045	14.217
VLRB	1459.743	0.7973	18.271

through an activation function. In this paper, we experimented with varying the number of neurons, ranging from 5 to 30.

As for the nature of the dataset and the natural behavior of ML-based methods, the results of the models at each time of simulation varies. To make a fair comparison, the models have to be run several times with the tuned hyperparameter while in each iteration, the randomized selection of the dataset during train-test splitting varies. To check the stability of the model, we run the model 50 times with tuned hyperparameters to make sure that the results we report are repeatable and reproducible. After doing this repetitive process, the mean performance of these iterations is reported.

The outcomes of this phase of the study, which are provided in table 2, revealed that the optimal number of neurons in the hidden layers is 20 for single, quadruple, and quintuple configurations, 30 for double configurations, and 10 for triple-layer models. The results from tuning the number of neurons indicate that the five-layer model with a setup of [20 20 20 20 20] neurons achieved the highest accuracy. This model exhibits the lowest MARE of 5.2802%, respectively, while has the highest R-squared of 86.59%.

The training function/algorithm is another important parameter that can be controlled in CFNN models. It is also referred to as an optimization function in the literature and primarily aims to minimize the error between the actual and predicted values. The objective of enhancing the model's performance involves adjusting its weight and bias factors after each training cycle. There is a wide range of training functions that have been developed by researchers. Therefore, an excellent choice of this function is extremely necessary as it directly influences the way that the model updates the neurons to reach a prediction near to experimental data. In this paper, we have tested six different training algorithms (i.e., Levenberg–Marquardt (LM, Scaled Conjugate Gradient (SCG), Fletcher-Powell Conjugate Gradient (FPCG), Polak-Ribiére Conjugate Gradient (PRCG), Resilient Backpropagation (RB), and Variable Learning Rate Backpropagation (VLRB)), which are frequently used in the literature and performed good predictability. The results of this step of the study are summarized in table 3 and show that the LM with 0.8658 R-squared value and 672.8319 RMSE value has the highest accuracy among these six functions.

The activation function serves to add non-linearity to the network to assist in grasping complex patterns in the data by transforming the weighted sum of inputs into a non-linear output. As was mentioned earlier in section 2.2.1, activation functions determine whether a neuron should be activated or remain inactive based on its input. When the input surpasses a specific threshold, the neuron activates, producing a non-zero output. If the input is below this threshold, the neuron stays inactive, resulting in a zero output. Therefore, the number of activation functions in each model is directly related to the number of hidden layers. By considering the number of hidden layers as m, the model has $m + 1$ activation functions that can be the same or different from each other.

There are several types of activation functions that have been used in literature. In this paper, we assessed four methods that are more common and performed satisfactory results. Among the selected options, purelin (pure linear) is a linear function, while the remaining ones including tansig (hyperbolic tangent sigmoid), satlin (saturated linear), and logsig (log-sigmoid) are non-linear functions. As can be seen in table 4, the Logsigmoid-Purelinear pair of activation functions demonstrates higher performance with 89.32% goodness of fit.

Table 4. Assessment of the effect of different activation function pairs on the CFNN model performance.

Activation function			Model performance		
Between input and hidden layers	Between two hidden layers	Between hidden and output layers	RMSE	R-squared	Testing time [ms]
Logsig	Logsig	Logsig	848.63	0.1615	8.14
Logsig	**Logsig**	**Purelin**	**410.18**	**0.8932**	**7.70**
Logsig	Logsig	Satlin	1040.01	0.1279	7.74
Logsig	Logsig	Tansig	16592.59	0.3981	13.00
Purelin	Purelin	Logsig	1280.00	0.0962	7.43
Purelin	Purelin	Purelin	576.38	0.5881	8.04
Purelin	Purelin	Satlin	24928.61	0.2249	11.20
Purelin	Purelin	Tansig	19501.25	0.2981	14.30
Satlin	Satlin	Logsig	838.98	0.1299	8.13
Satlin	Satlin	Purelin	1410.40	0.8038	8.24
Satlin	Satlin	Satlin	1090.31	0.0821	7.73
Satlin	Satlin	Tansig	653.42	0.7977	8.21
Tansig	Tansig	Logsig	882.37	0.1414	8.15
Tansig	Tansig	Purelin	675.03	0.8431	8.71
Tansig	Tansig	Satlin	1210.12	0.1585	7.27
Tansig	Tansig	Tansig	1140.51	0.5944	18.30

As of now the model with 5 hidden layers demonstrated the highest accuracy, there are six activation functions in the model. Testing all possible configurations of these functions needs a huge amount of time and computation resources. To avoid this, it is quite common among ML researchers to consider a pair of these functions and test their effect on the accuracy of the model. The first member of these pairs is considered as the activation function between the input and hidden layers and also between the hidden layers themselves. The second pair is therefore considered for functioning between the hidden and output layers.

As a result, the quintuple hidden layers model that has 20 neurons in each of its hidden layers and used the LM algorithm for training of the model and [Logsig-Logsig-Purelin] activation functions have the ability to predict the CHF with an accuracy of 89.32% in terms of R-squared.

3.2. XGBoost

There are some important hyperparameters that can effectively change the accuracy of the XGBoost model. Since it is an ensemble method, unlike the CFNN, most of these parameters are for controlling the decision trees within XGBoost. The parameters that have been studied in this paper are: the number of boosting rounds, maximum depth, and learning rate.

The number of boosting rounds or trees that are built in a model, which is also known as the number of estimators, is one of the most important hyperparameters for the XGBoost method. Generally, the higher the number of boosting rounds, the more powerful the model is. That said, too many boosting rounds would lead to overfitting conditions, which must be avoided. Moreover, depending on the dataset, an optimal number of trees should be chosen. Increasing the number of trees enhances performance until reaching a certain threshold, after which notable improvements may not occur, and there could be an increase in training time. In this work, the range of variation for this hyperparameter is considered between 10 to 200 as for the values above 120, the results were stable and were not affected by changing the value of this parameter. Looking closely at figure 6, The graph indicates that an initial sharp increase in R-squared with the increase in the number of estimators, which reaches a peak very quickly at 30 estimators. After this peak, there's a sharp decline in R-squared as the number of estimators continues to increase, suggesting that the optimal number of estimators may be just before this drop. Following the decline, the R-squared value seems to stabilize, indicating that additional estimators beyond this point do not significantly change the performance of the model. This plateau suggests that the model has reached a point where additional complexity (in the form of more estimators) does not yield better predictions. The significant initial peak might indicate that a relatively small number of estimators is sufficient for the model to capture the underlying data structure. The subsequent decline and stabilization suggest that adding more estimators does not necessarily lead to better performance and could be a sign of overfitting.

The maximum depth of a tree is another hyperparameter for XGBoost. Increasing the depth of trees enables them to grasp more intricate patterns in the data, but it also raises the possibility of overfitting. Selecting the right maximum depth is vital when the XGBoost method is used. On the one hand, if it is too small, the model might be overly simplistic and miss important information in the data. On the other hand, if it is too large, the model could memorize the training data and not perform well on new unseen data. The minimum value for this

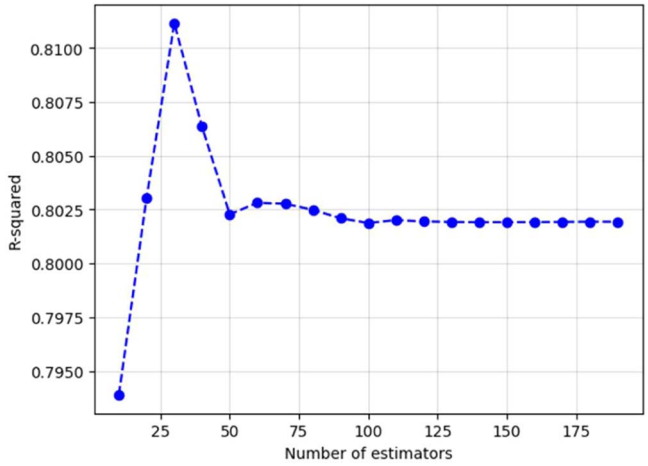

Figure 6. Sensitivity analysis of the performance of the XGBoost model on the changes in the number of estimators.

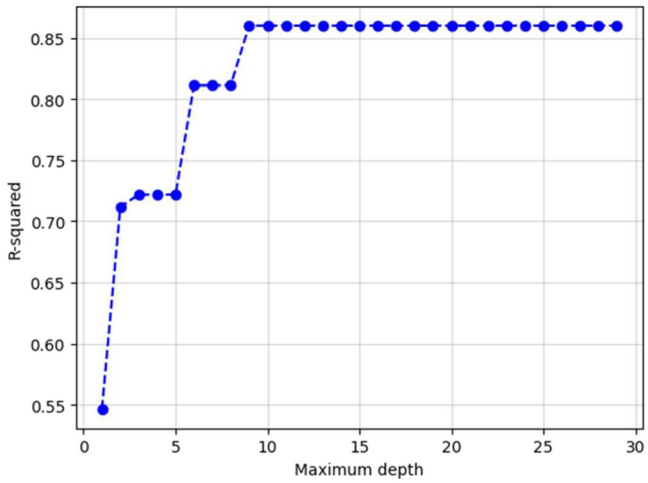

Figure 7. Sensitivity analysis of the performance of the XGBoost model on the changes in the maximum depth of trees.

parameter is 2. In this paper, a range between 2 and 30 was selected for the tuning process as there were no changes in the performance of the model above this range. As it appears from figure 7, a rapid increase in R-squared as the maximum depth goes from 1 to 9, indicating that deeper trees initially provide a better fit to the data. After this initial rise, the R-squared value plateaus, and the performance stabilizes despite further increases in the tree depth. This suggests that there is little to no benefit in terms of R-squared from making the trees any deeper beyond the maximum depth of 9, which is suggested as the best value in the analysis.

The learning rate guides how large each step is during the model's progress toward minimizing the loss function. It is a crucial setting that affects how quickly the model reaches a solution and its overall performance. A lower learning rate needs more boosting rounds (trees) to achieve the same accuracy, but it can make the model more dependable. Conversely, a higher learning rate might make the model converge faster, but it raises the risk of overshooting the minimum or even going off course [94]. Typically, the learning rate is set between 0 and 1. A common starting point is 0.1, and it can be fine-tuned based on trial and error and the specific characteristics of the dataset. As is evidenced by figure 8, at the beginning, as the learning rate increases from a low value, there is a rapid improvement in the model's performance as indicated by the rising R-squared value. The model reaches a plateau in R-squared values relatively quickly, suggesting that there's an optimal range of learning rates that allows the model to perform well. Further increases in the learning rate within this optimal range do not significantly impact performance. Beyond a certain point, as the learning rate continues to increase, the performance becomes more volatile, with R-squared values varying more dramatically. This can be indicative of the model potentially overfitting or not being able to generalize well due to the learning rate being too high, which could lead the model to miss the global minimum of the loss function.

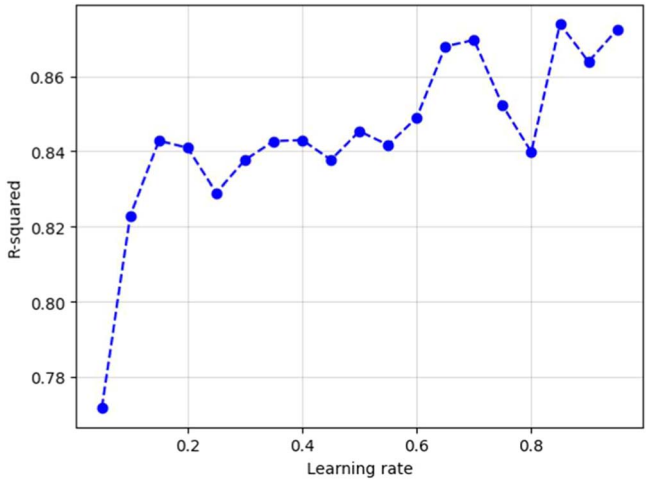

Figure 8. Sensitivity analysis of the performance of the XGBoost model on the changes in the learning rate.

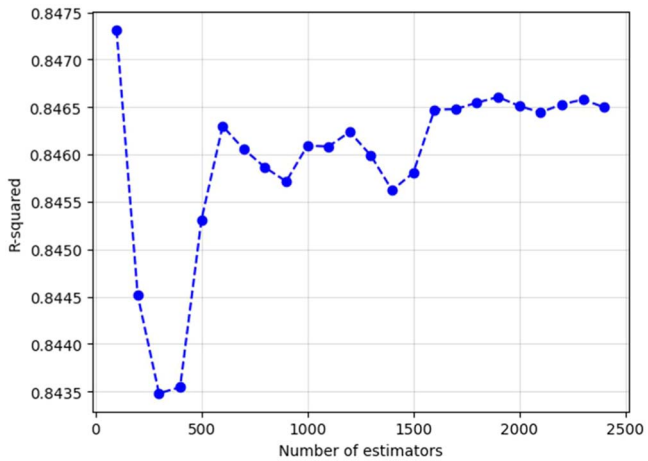

Figure 9. Sensitivity analysis of the performance of the Extra tree model on the changes in the number of estimators.

3.3. Extra tree

Similar to XGBoost, the Extra tree is an ensemble model and has common hyperparameters. Same as what was explained in section 3.2, the number of estimators and maximum depth of trees have been tuned for Extra trees too. But the difference is that the effective range of values for the number of estimators differs. For this parameter, a range of values from 100 to 2500 is considered as the performance of the model was steady above this range. As depicted in figure 9, the best choice for the number of estimators would be 100 as resulted in the highest accuracy in terms of R-squared. Starting from a low number of estimators, there is a sharp increase in R-squared, indicating that initially adding more trees improves the model's performance significantly. After reaching a peak, there's a sharp drop, which suggests that there might be an optimal number of estimators beyond which adding more trees decreases performance, potentially due to overfitting or instability at that particular configuration. The R-squared value then recovers and stabilizes, fluctuating around a certain level as the number of estimators continues to increase. From the plateau in the R-squared values, it seems that adding more estimators beyond a certain point does not lead to significant improvements in model performance, indicating that there is a region where the model achieves a balance between bias and variance. Also, figure 10 provides insights regarding the change in accuracy of the model with respect to the changes in maximum depth of the trees. As it appears from this figure, by considering a value beyond 20 for this hyperparameter, no notable changes will be made. So, this value can be considered as the best value within this range to improve the performance of the model for the next steps.

Another parameter that has considerable influence on the accuracy of the model is the minimum number of samples for splitting. This hyperparameter sets the minimum number of samples that are required to split an

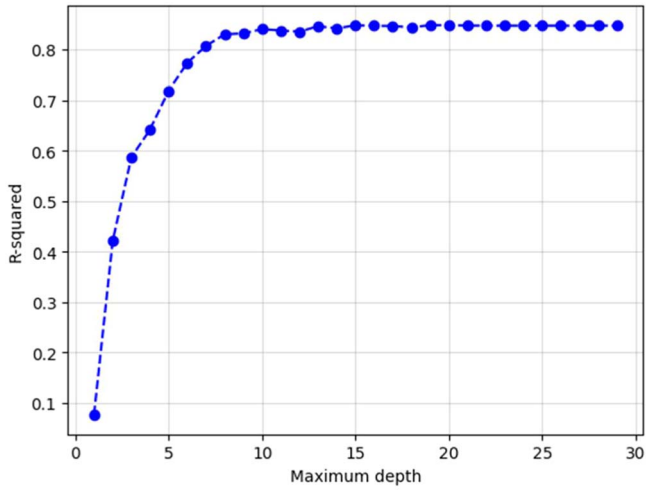

Figure 10. Sensitivity analysis of the performance of the Extra tree model on the changes in the maximum depth of trees.

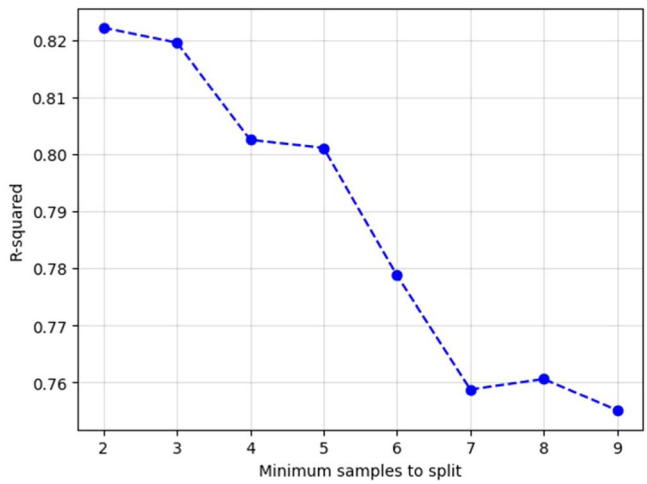

Figure 11. Sensitivity analysis of the performance of the Extra tree model on the changes of the minimum number of samples for splitting.

internal node in a decision tree. If the number of samples at a node is less than the pre-set value, the node will not be split, and it will become a leaf node. Increasing this value can prevent the algorithm from creating splits that are too specific to the training data, which helps in reducing the chance of overfitting conditions. In other words, it encourages the algorithm to make splits only when there is sufficient data at a node. A common range for this parameter in the literature is between 2 and 10, which is considered in this paper too. According to information in figure 11, as the minimum number of samples required for a split increases from 2 to around 5, there is a slight decrease in the R-squared value. This suggests that being too flexible (allowing splits with few samples) does not necessarily degrade the model performance significantly within this range. There's a more pronounced decrease in R-squared as the threshold is increased beyond 5 samples. This could indicate that requiring too many samples to allow a split may prevent the model from learning finer details in the data, leading to underfitting. The graph also suggests that there's an optimal range for this hyperparameter that balances the model's ability to learn from the data without overfitting. In this case, that might be around 2 to 4 samples.

3.4. LightGBM

The hyperparameters of LightGBM that highly affect the performance of the model are the number of leaves and the learning rate. The number of leaves defines the maximum number of leaves (terminal nodes) that can be formed in each tree. In other words, it controls the complexity of the tree model. The number of leaves is a crucial factor because it directly influences the model's capacity to learn the data. A higher number of leaves allows the model to learn more fine-grained patterns and increase its complexity, potentially leading to a better

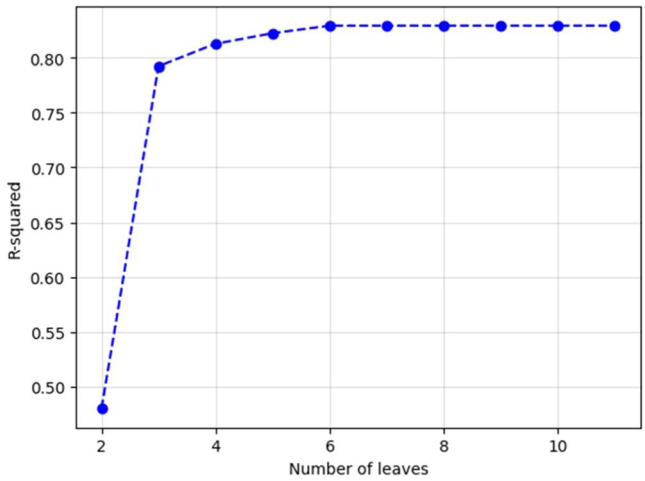

Figure 12. Sensitivity analysis of the performance of the LightGBM model on the changes of the minimum number of leaves.

fit on the training data. However, this can also make the model more prone to overfitting. As is demonstrated in figure 12, as the number of leaves increases, the R-squared value sharply rises, suggesting that allowing more leaves in the trees significantly improves the model's fit to the data initially. After the initial sharp increase, the R-squared value plateaus, indicating that there is a point beyond which adding more leaves does not result in a substantial increase in model performance. This suggests that the model has reached a balance in its complexity where it is sufficient to capture the trends in the data without additional complexity. The flat part of the curve suggests that there is a range of 'number of leaves' values for which the LightGBM model will perform consistently. Choosing a number of leaves within this range might provide the best generalization on unseen data, preventing overfitting while still maintaining a high R-squared score.

The learning rate parameter, the same as XGBoost, controls the step size at each iteration while moving toward a minimum of the loss function. In essence, it determines how much the model is adjusted in response to the error each tree make slowerller learning rate requires more trees to model all the relationships in the data, leading to a more complex model that may perform better but will take longer to train. Conversely, a larger learning rate makes larger adjustments to the model with each tree, potentially leading to a less accurate model that trains faster.

According to figure 13, A general decline in R-squared as the learning rate increases, indicating that the model's ability to predict the variance of the dependent variable decreases. The performance fluctuates with changes in the learning rate, showing peaks and troughs. It's not a smooth curve, which could imply that certain learning rates are more optimal for the model than others. Also, the most notable drop in R-squared happens after the learning rate exceeds 0.6, suggesting that higher learning rates may lead to poorer performance for this particular model. Moreover, there is volatility in model performance, especially as the learning rate increases, which might suggest overfitting at certain learning rates or instability in the model's learning process.

3.5. Results comparison and discussion

Now, the tuned models need to be compared with each other using the performance indexes. To take the effect of dataset random splitting into account, the training-testing process of the best model that resulted from the sensitivity analysis repeated 50 times, changing the way that the input dataset is split for the training and testing process. This study which is known as stability analysis, helps us to understand which model's performance is more stable. Table 5 provides the results of the iterative study for the tuned models of each of the CFNN, XGBoost, and Extra tree techniques. In this table, the mean value helps to understand the overall accuracy of the model while the standard deviation shows how the accuracy of the model is dependent to the training and testing datasets. The lower value of Std means that the technique is more robust and is not much affected by changing training dataset selection. As can be seen in table 5, the CFNN model has better goodness of fit rather compared with XGBoost and Extra tree models. This means that the predictability of this method is better than the other ones. However, it has a higher RMSE on average, which shows that for some datapoints, it has large errors. This is because of that the RMSE index escalates/magnifies the effect of large errors.

Also, table 6 compares the performance of the proposed models with the existing AI-based models published in the literature based on R-squared. As can be seen from table 6, the mean R-squared of all four proposed models in this paper are higher than existing models in the literature, which were based on Support Vector

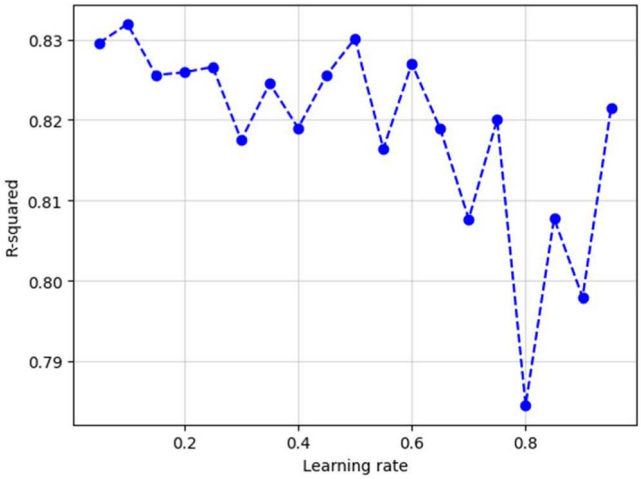

Figure 13. Sensitivity analysis of the performance of the LightGBM model on the changes in the learning rate.

Table 5. Summary of performance of tuned models of four implemented techniques.

Method	Mean RMSE	Std RMSE	Mean R-squared	Std R-squared
CFNN	410.189273	148.571	0.89322	0.03483
XGBoost	215.578393	46.7618	0.88066	0.05141
Extra tree	222.563672	41.8633	0.87101	0.06122
LightGBM	320.616516	65.583040	0.72623	0.15670

Table 6. Stability of the performance of the models of present work and existing models in the literature in terms of R-squared.

Source	Method	Mean	Std	MARD
[21]	SVR	0.7433	0.0602	0.0491
[21]	MLP	0.8261	0.0336	0.0243
[21]	RF	0.7698	0.1056	0.0711
Present paper	CFNN	0.8932	0.0348	0.0268
Present paper	XGBoost	0.8807	0.0514	0.0419
Present paper	Extra tree	0.8710	0.0612	0.0455
Present paper	LightGBM	0.7262	0.1567	0.0912

Regression (SVR), Multilayer perceptron (MLP), and Random Forest (RF). Also, while CFNN model has almost as same stability as MLP, it can predict the CHF with approximately 7% better accuracy.

To clarify the innovative aspects and unique contributions of our research, it is worth mentioning the following points regarding the results of this work:

1. Integration of Complex Data Sets: Our AI-based models are designed to integrate and analyze complex datasets which include varied nanoparticle sizes, types, and concentrations in nanofluids. This capability to handle high-dimensional data (several input/effective parameters) far exceeds traditional regression models that often fail to capture such complexity.

2. Advanced AI Algorithms for Enhanced Predictability: We have utilized advanced AI algorithms such as CFNN, XGBoost, and Extra Trees. These models can identify nonlinear relationships and intricate patterns that are not apparent through traditional methods, leading to more accurate predictions of critical heat flux.

3. Scalability and Robustness: Our AI models demonstrate exceptional scalability and robustness, handling large datasets efficiently, a significant advantage over conventional methods that struggle with scale and complexity. This scalability is crucial for enhancing predictive accuracy as the volume of experimental data grows.

4. Generalization Capabilities: The AI techniques employed in our study have shown superior generalization capabilities across various data patterns and conditions. This is especially valuable in applications where operational environments can drastically vary, ensuring that our models remain effective in unseen scenarios.
5. Model Adaptability and Fine-tuning: Unlike traditional fitting methods, our AI-based models can be continuously fine-tuned and re-trained as new data becomes available or when different nanofluid formulations are used. This adaptability makes our approach highly versatile and practical for ongoing applications in thermal management systems.

The unique contribution of our work lies in developing and benchmarking several intelligent models based on these sophisticated AI methodologies specifically to the challenge of predicting critical heat flux in nanofluids. This not only provides a theoretical advancement in the thermal physics of nanofluids but also offers practical implications for the design and optimization of cooling systems in various industrial applications, significantly improving safety and efficiency.

4. Conclusion

Enhancing the heat transfer of the thermal energy exchanging systems is paramount to increase the overall efficiency of these systems. In the systems that are working under pool boiling conditions, CHF is a critical limit for the system to dissipate the thermal energy from hotter surfaces. Implementing nanofluid instead of pure fluids can enhance the CHF. There are many different factors that influence the CHF of a nanofluid such as the temperature, pressure, nano-particle size material etc, making the pattern of data for CHF extensively intricate. Because of this, traditional data-driven modelling ways like mathematical fittings can no longer be useful for modelling CHF of nanofluids, if the accuracy of results is important. Therefore, it is necessary to implement more robust methods like AI techniques. In this paper, four AI models, i.e., CFNN, XGBoost, LightGBM, and Extra tree methods based on experimental data have been developed to predict the CHF of the nanofluids under different properties of nanoparticles and working conditions. This will help the designers of these systems to accurately estimate the CHF of the nanofluid that they want to implement in the system instead of conducting extra experiments that can cost a great deal of money for the facilities that are needed.

The findings of this paper can be summarized as follows:

- The CFNN model demonstrated the highest accuracy amongst the four developed with 89.32% of R-squared.
- The XGBoost and Extra tree methods are the second and third accurate models, respectively and LightGBM has the worst performance among all developed models in this paper.
- The CFNN method had a lower Std of R-squared that shows the results of the model are more stable and the performance of the model does not fluctuate significantly when the training dataset changes.
- All four developed models in this paper are more accurate than existing models in the literature.

Acknowledgments

For the purpose of open access, the author(s) has applied a Creative Commons Attribution (CC BY) license to any Author Accepted Manuscript version arising from this submission.

Data availability statement

All data that support the findings of this study are included within the article (and any supplementary files).

ORCID iDs

Shahin Alipour Bonab https://orcid.org/0009-0002-8316-4336
Mohammad Yazdani-Asrami https://orcid.org/0000-0002-7691-3485

References

[1] Tatsumoto H *et al* 2014 Forced convection heat transfer of saturated liquid hydrogen in vertically-mounted heated pipes *AIP Conf. Proc.* (American Institute of Physics Inc) 44–51

[2] Kwark S M, Amaya M, Kumar R, Moreno G and You S M 2010 Effects of pressure, orientation, and heater size on pool boiling of water with nanocoated heaters *Int. J. Heat Mass Transf.* **53** 5199–208

[3] Tatsumoto H *et al* 2012 Forced convection heat transfer of liquid hydrogen through a 200-mm long heated tube *In Physics Procedia* (Elsevier B.V) 1360–5

[4] Shirai Y *et al* 2011 Forced flow boiling heat transfer of liquid hydrogen for superconductor cooling *Cryogenics* **51** 295–9

[5] Shirai Y *et al* 2018 DNB heat flux on inner side of a vertical pipe in forced flow of liquid hydrogen and liquid nitrogen *Cryogenics (Guildf)* **92** 105–17

[6] Kuang Y, Han F, Sun L, Zhuan R and Wang W 2021 Modeling and numerical investigation of hydrogen nucleate flow boiling heat transfer *Int. J. Hydrogen Energy* **46** 19617–32

[7] Baldwin M, Ghavami A, Ghiaasiaan S M and Majumdar A 2021 Pool boiling in liquid hydrogen, liquid methane and liquid oxygen: a review of available data and predictive tools *Cryogenics (Guildf)* **115** 103240

[8] Sakashita H 2016 Pressure effect on CHF enhancement in pool boiling of nanofluids *J. Nucl. Sci. Technol.* **53** 797–802

[9] Kandlikar S G 2001 A Theoretical model to predict pool boiling chf incorporating effects of contact angle and orientation *J. Heat Transfer* **123** 1071–9

[10] Wang J *et al* 2021 Numerical investigation on subcooled pool film boiling of liquid hydrogen in different gravities *Int. J. Hydrogen Energy* **46** 2646–57

[11] Bi J, Vafai K and Christopher D M 2015 Heat transfer characteristics and CHF prediction in nanofluid boiling *Int. J. Heat Mass Transf.* **80** 256–65

[12] Wang L *et al* 2022 Experimental study on the CHF enhancement effect of nanofluids on the oxidized low carbon steel surface *Appl. Therm. Eng.* **204** 117968

[13] Yazdani-Asrami M, Sadeghi A and Atrey M D 2022 Selecting a cryogenic cooling system for superconducting machines: general considerations for electric machine designers and engineers *Int. J. Refrig* **140** 70–81

[14] do Nascimento F J, Moreira T A and Ribatski G 2019 Flow boiling critical heat flux of DI-water and nanofluids inside smooth and nanoporous round microchannels *Int. J. Heat Mass Transf.* **139** 240–53

[15] Datta A and Halder P 2023 Field-synergy and nanoparticle's diameter analysis on circular jet impingement using three oxide–water-based nanofluids *Journal of Thermal Engineering* **9** 179–90

[16] Jaiswal A, Datta A and Halder P 2018 Heat transfer augmenation using nanofluid of a confined slot impingement jet on concave surface *IOP Conf. Ser.: Mater. Sci. Eng.* **377** 012075

[17] Datta A and Halder P 2021 Thermal efficiency and hydraulic performance evaluation on Ag–Al2O3 and SiC–Al2O3 hybrid nanofluid for circular jet impingement *Archives of Thermodynamics* **42** 163–82

[18] Datta A, Kumar S and Halder P 2020 Heat transfer and thermal characteristics effects on moving plate impinging from Cu-water nanofluid Jet *J. Therm. Sci.* **29** 182–93

[19] Datta A, Jaiswal A and Halder P 2018 Heat transfer analysis of slot jet impingement using Nano fluid on convex surface *IOP Conf. Ser.: Mater. Sci. Eng.* **402** 012098

[20] Yazdani-Asrami M, Fang L, Pei X and Song W 2023 Smart fault detection of HTS coils using artificial intelligence techniques for large-scale superconducting electric transport applications *Supercond. Sci. Technol.* **36** 085021

[21] Serrao B P, Kim K M and Duarte J P 2023 Analysis of the effects of different nanofluids on critical heat flux using artificial intelligence *Energies (Basel)* **16** 4762

[22] Yazdani-Asrami M, Taghipour-Gorjikolaie M, Song W, Zhang M, Chakraborty S and Yuan W 2021 Artificial intelligence for superconducting transformers *Transformers Magazine* **8** 22–30 Accessed: Jan. 20, 2024. [Online]. Available www.transformers-magazine.com

[23] Swartz B, Wu L, Zhou Q and Hao Q 2021 Machine learning predictions of critical heat fluxes for pillar-modified surfaces *Int. J. Heat Mass Transf.* **180** 121744

[24] Erturk A S, Malakizadi A and Larsson R 2023 Towards an accurate estimation of heat flux distribution in metal cutting by machine learning *In Procedia CIRP* (Elsevier B.V) 359–64

[25] Xia J S *et al* 2023 Using feed-forward perceptron artificial neural network (ANN) model to determine the rolling force, power and slip of the tandem cold rolling *ISA Trans.* **132** 353–63

[26] Hema M, Toghraie D and Amoozad F 2023 Prediction of viscosity of MWCNT-Al2O3 (20:80)/SAE40 nano-lubricant using multi-layer artificial neural network (MLP-ANN) modeling *Eng. Appl. Artif. Intell.* **121** 105948

[27] Esfe M H *et al* 2023 A novel integrated model to improve the dynamic viscosity of MWCNT-Al2O3 (40:60)/Oil 5W50 hybrid nano-lubricant using artificial neural networks (ANNs) *Tribol. Int.* **178** 108086

[28] Hemmat Esfe M, Ali Eftekhari S, Alizadeh A, Aminian S, Hekmatifar M and Toghraie D 2023 A well-trained artificial neural network for predicting the optimum conditions of MWCNT–ZnO (10:90)/ SAE 40 nano-lubricant at different shear rates, temperatures, and concentration of nanoparticles *Arabian J. Chem.* **16** 104508

[29] Esfe M H, Toghraie D and Amoozadkhalili F 2023 Increasing the accuracy of estimating the viscosity of SAE40-based nanofluid containing MWCNT-TiO2 nanoparticles using a creative method in optimizing the structure of the artificial neural network *Tribol. Int.* **179** 108161

[30] Hua S *et al* 2023 Dynamic viscosity prediction using artificial intelligence for an antifreeze containing MWCNT–alumina hybrid nanopowders *Eng. Appl. Artif. Intell.* **126** 107046

[31] Alizadeh A *et al* 2023 Evaluation of the effects of the presence of ZnO -TiO2 (50%–50%) on the thermal conductivity of Ethylene Glycol base fluid and its estimation using Artificial Neural Network for industrial and commercial applications *Journal of Saudi Chemical Society* **27** 101613

[32] Alipour Bonab S, Sadeghi A and Yazdani-Asrami M 2024 Artificial intelligence-based surrogate model for computation of the electric field of high voltage transmission line ceramic insulator with corona ring *World Journal of Engineering* (https://doi.org/10.1108/WJE-11-2023-0478)

[33] Yazdani-Asrami M *et al* 2023 Roadmap on artificial intelligence and big data techniques for superconductivity *Supercond. Sci. Technol.* **36** 043501

[34] Al-Khasawneh M A, Bukhari A and Khasawneh A M 2022 Effective of smart mathematical model by machine learning classifier on big data in healthcare fast response *Comput. Math Methods Med.* **2022** 1–9

[35] Sun K, Xiao H, Liu S and Liu Y 2020 Machine learning-based fast frequency response control for a VSC-HVDC system *CSEE Journal of Power and Energy Systems* **7** 688–97
[36] Couto Carneiro A M, Carvalho Alves A F, Cardoso Coelho R P, Cardoso J S and Andrade Pires F M 2023 A simple machine learning-based framework for faster multi-scale simulations of path-independent materials at large strains *Finite Elem. Anal. Des.* **222**
[37] Mehralizadeh A, Shabanian S R and Bakeri G 2020 Experimental and modeling study of heat transfer enhancement of TiO2/SiO2 hybrid nanofluids on modified surfaces in pool boiling process *The European Physical Journal Plus* **135** 796
[38] Faridzadeh M R, Semiromi D T and Niroomand A 2014 Analysis of laminar mixed convection in an inclined square lid-driven cavity with a nanofluid by using an artificial neural network *Heat. Transf. Res.* **45** 361–90
[39] Rostami S, Toghraie D, Esfahani M A, Hekmatifar M and Sina N 2021 Predict the thermal conductivity of SiO2/water–ethylene glycol (50:50) hybrid nanofluid using artificial neural network *J. Therm. Anal. Calorim.* **143** 1119–28
[40] Kim H, Moon J, Hong D, Cha E and Yun B 2021 Prediction of critical heat flux for narrow rectangular channels in a steady state condition using machine learning *Nuclear Engineering and Technology* **53** 1796–809
[41] He M and Lee Y 2018 Application of machine learning for prediction of critical heat flux: Support vector machine for data-driven CHF look-up table construction based on sparingly distributed training data points *Nucl. Eng. Des.* **338** 189–98
[42] Rivera J, San Martin D, Gollner M, Torres C E and Fernandez-Pello C 2023 A machine learning approach to predict the critical heat flux for ignition of solid fuels *Fire Saf. J.* **141**
[43] Zhang J, Zhong D, Shi H, Meng J and Chen L 2022 Machine learning prediction of critical heat flux on downward facing surfaces *Int. J. Heat Mass Transf.* **191** 122857
[44] Zhao X, Shirvan K, Salko R K and Guo F 2020 On the prediction of critical heat flux using a physics-informed machine learning-aided framework *Appl. Therm. Eng.* **164** 114540
[45] Yan S R, Toghraie D, Abdulkareem L A, Alizadeh A, Barnoon P and Afrand M 2020 The rheological behavior of MWCNTs–ZnO/Water–Ethylene glycol hybrid non-Newtonian nanofluid by using of an experimental investigation *Journal of Materials Research and Technology* **9** 8401–6
[46] Khodabandeh E, Rozati S A, Joshaghani M, Akbari O A, Akbari S and Toghraie D 2019 Thermal performance improvement in water nanofluid/GNP–SDBS in novel design of double-layer microchannel heat sink with sinusoidal cavities and rectangular ribs *J. Therm. Anal. Calorim.* **136** 1333–45
[47] Al-Kouz W *et al* 2021 Heat transfer and entropy generation analysis of water-Fe3O4/CNT hybrid magnetic nanofluid flow in a trapezoidal wavy enclosure containing porous media with the Galerkin finite element method *Eur. Phys. J. Plus* **136**
[48] Stange G, Yeom H, Semerau B, Sridharan K and Corradini M 2013 A study of nanoparticle surface modification effects on pool boiling critical heat flux *Nucl. Technol.* **182** 286–301
[49] Zuhairi Sulaiman M, Matsuo D, Enoki K and Okawa T 2016 Systematic measurements of heat transfer characteristics in saturated pool boiling of water-based nanofluids *Int. J. Heat. Mass Transf.* **102** 264–76
[50] Golubovic M N, Madhawa Hettiarachchi H D, Worek W M and Minkowycz W J 2009 Nanofluids and critical heat flux, experimental and analytical study *Appl. Therm. Eng.* **29** 1281–8
[51] Lee J H, Lee T and Jeong Y H 2012 Experimental study on the pool boiling CHF enhancement using magnetite-water nanofluids *Int. J. Heat Mass Transf.* **55** 2656–63
[52] Kim S J, Bang I C, Buongiorno J and Hu L W 2007 Surface wettability change during pool boiling of nanofluids and its effect on critical heat flux *Int. J. Heat Mass Transf.* **50** 4105–16
[53] Ham J, Kim H, Shin Y and Cho H 2017 Experimental investigation of pool boiling characteristics in Al2O3 nanofluid according to surface roughness and concentration *Int. J. Therm. Sci.* **114** 86–97
[54] Vazquez D M and Kumar R 2013 Surface effects of ribbon heaters on critical heat flux in nanofluid pool boiling *Int. Commun. Heat Mass Transfer* **41** 1–9
[55] Lee J H, Lee T and Jeong Y H 2013 The effect of pressure on the critical heat flux in water-based nanofluids containing Al2O3 and Fe3O4 nanoparticles *Int. J. Heat Mass Transf.* **61** 432–8
[56] Song S L, Lee J H and Chang S H 2014 CHF enhancement of SiC nanofluid in pool boiling experiment *Exp. Therm Fluid Sci.* **52** 12–8
[57] Jung J-Y, Kim H and Kim M H 2013 Effect of ionic additive on pool boiling critical heat flux of titania/water nanofluids *Heat Mass Transfer* **49** 1–10
[58] Kim H and Kim M H 2006 Experimental study on CHF characteristics of water-Tio2 nano-fluids [Online]. Available https://researchgate.net/publication/264176856
[59] Kshirsagar J M and Shrivastava R 2018 Experimental investigation of nucleate pool boiling characteristics of high concentrated alumina/water nanofluids *Heat and Mass Transfer/Waerme- und Stoffuebertragung* **54** 1779–90
[60] Jung J Y, Kim E S and Kang Y T 2012 Stabilizer effect on CHF and boiling heat transfer coefficient of alumina/water nanofluids *Int. J. Heat Mass Transf.* **55** 1941–6
[61] Hegde N R, Rao S S and Reddy R 2012 Flow visualization and study of critical heat flux enhancement in pool boiling with Al 2O 3-water nanofluids *Thermal Science* **16** 445–53
[62] Kim E S, Jung J-Y and Kang Y T 2013 The effect of surface area on pool boiling heat transfer coefficient and CHF of Al2O3/water nanofluids *J. Mech. Sci. Technol.* **27** 3177–82
[63] Shahmoradi Z, Etesami N and Nasr Esfahany M 2013 Pool boiling characteristics of nanofluid on flat plate based on heater surface analysis *Int. Commun. Heat Mass Transfer* **47** 113–20
[64] Kumar N, Urkude N, Sonawane S S and Sonawane S H 2018 Experimental study on pool boiling and Critical Heat Flux enhancement of metal oxides based nanofluid *Int. Commun. Heat Mass Transfer* **96** 37–42
[65] Yao S, Huang X, Song Y, Shen Y and Zhang S 2018 Effects of nanoparticle types and size on boiling feat transfer performance under different pressures *AIP Adv.* **8**
[66] Bang I C and Heung Chang S 2005 Boiling heat transfer performance and phenomena of Al2O3–water nano-fluids from a plain surface in a pool *Int. J. Heat Mass Transf.* **48** 2407–19
[67] Ciloglu D 2017 An experimental investigation of nucleate pool boiling heat transfer of nanofluids from a hemispherical surface *Heat Transfer Eng.* **38** 919–30
[68] Rostamian F and Etesami N 2018 Pool boiling characteristics of silica/water nanofluid and variation of heater surface roughness in domain of time *Int. Commun. Heat Mass Transfer* **95** 98–105
[69] Dadjoo M, Etesami N and Esfahany M N 2017 Influence of orientation and roughness of heater surface on critical heat flux and pool boiling heat transfer coefficient of nanofluid *Appl. Therm. Eng.* **124** 353–61
[70] Kathiravan R, Kumar R, Gupta A and Chandra R 2010 Preparation and pool boiling characteristics of copper nanofluids over a flat plate heater *Int. J. Heat Mass Transf.* **53** 1673–81

[71] Kathiravan R, Kumar R, Gupta A and Chandra R 2009 Characterization and pool boiling heat transfer studies of nanofluids,' *J. Heat Transfer* **131** 1–8
[72] Hu Y, Liu Z and He Y 2018 Effects of SiO2 nanoparticles on pool boiling heat transfer characteristics of water based nanofluids in a cylindrical vessel *Powder Technol.* **327** 79–88
[73] Salari E, Peyghambarzadeh S M, Sarafraz M M and Hormozi F 2016 Boiling thermal performance of TiO2 aqueous nanofluids as a coolant on a disc copper block *Periodica Polytechnica Chemical Engineering* **60** 106–22
[74] Yang X F and Liu Z H 2011 Pool boiling heat transfer of functionalized nanofluid under sub-atmospheric pressures *Int. J. Therm. Sci.* **50** 2402–12
[75] Yagnem A R and Venkatachalapathy S 2019 Heat transfer enhancement studies in pool boiling using hybrid nanofluids *Thermochim. Acta* **672** 93–100
[76] Ahn H S and Kim M H 2013 The boiling phenomenon of alumina nanofluid near critical heat flux *Int. J. Heat Mass Transf.* **62** 718–28
[77] Warsito B, Santoso R, Suparti S and Yasin H 2018 Cascade forward neural network for time series prediction *J. Phys. Conf. Ser.* **1025** 012097
[78] Alkhasawneh M S 2019 Hybrid cascade forward neural network with elman neural network for disease prediction *Arab. J. Sci. Eng.* **44** 9209–20
[79] Alkhasawneh M S and Tay L T 2018 A hybrid intelligent system integrating the cascade forward neural network with elman neural network *Arab. J. Sci. Eng.* **43** 6737–49
[80] Alzayed M, Chaoui H and Farajpour Y 2021 Maximum power tracking for a wind energy conversion system using cascade-forward neural networks *IEEE Trans Sustain Energy* **12** 2367–77
[81] Alipour Bonab S, Song W and Yazdani-Asrami M 2023 A new intelligent estimation method based on the cascade-forward neural network for the electric and magnetic fields in the vicinity of the high voltage overhead transmission lines *Applied Sciences* **13** 11180
[82] Yazdani-Asrami M, Sadeghi A and Song W 2023 Ultra-fast surrogate model for magnetic field computation of a superconducting magnet using multi-layer artificial neural networks *J. Supercond. Nov. Magn.* **36** 575–86
[83] Yazdani-Asrami M, Sadeghi A, Seyyedbarzegar S and Song W 2022 DC electro-magneto-mechanical characterization of 2G HTS tapes for superconducting cable in magnet system using artificial neural networks *IEEE Trans. Appl. Supercond.* **32**
[84] Ituabhor O, Isabona J, Zhimwang J T and Risi I 2022 Cascade forward neural networks-based adaptive model for real-time adaptive learning of stochastic signal power datasets *International Journal of Computer Network and Information Security* **14** 63–74
[85] Chen T and Guestrin C 2016 XGBoost: a scalable tree boosting system 785–794
[86] Guo X *et al* 2023 Critical role of climate factors for groundwater potential mapping in arid regions: Insights from random forest, XGBoost, and LightGBM algorithms *J. Hydrol. (Amst)* **621** 129599
[87] Cao W, Liu Y, Mei H, Shang H and Yu Y 2023 Short-term district power load self-prediction based on improved XGBoost model *Eng. Appl. Artif. Intell.* **126** 106826
[88] Wan A, Gong Z, Chen T and AL-Bukhaiti K 2024 Mass flow characteristics prediction of refrigerants through electronic expansion valve based on XGBoost *Int. J. Refrig* **158** 345–52
[89] Geurts P, Ernst D and Wehenkel L 2006 Extremely randomized trees *Mach. Learn.* **63** 3–42
[90] Yang H, Wang J, Wen J and Xie H 2023 Assessment of machine learning models and conventional correlations for predicting heat transfer coefficient of liquid hydrogen during flow boiling *Int. J. Hydrogen Energy* **49** 753–70
[91] Simm J, Magrans De Abril I and Sugiyama M 2014 Tree-based ensemble multi-task learning method for classification and regression *IEICE Trans. Inf. Syst.* **E97-D** 1677–81
[92] Yazdani-Asrami M *et al* 2022 Artificial intelligence methods for applied superconductivity: material, design, manufacturing, testing, operation, and condition monitoring *Supercond. Sci. Technol.* **35** 123001
[93] He Y *et al* 2022 Data-driven approach to predict the flow boiling heat transfer coefficient of liquid hydrogen aviation fuel *Fuel* **324** 124778
[94] 'XGBoost Parameters — xgboost 2.0.3 documentation 2024 Accessed [Online]. Available https://xgboost.readthedocs.io/en/stable/parameter.html